Record Synchronization Engine – Design Summary

- Record Synchronization Engine – Design Summary
  - 1. Problem Framing & Scope
  - 2. High-Level Design
  - 3. Core Synchronization Flow
    - 3.1 Outbound Sync
    - 3.2 Outbound Sync
  - 4. Key Design Decisions & Trade-offs
  - 5. Assumptions & Future Enhancements
    - Assumptions
    - Future Enhancements
  - 6. How to Run
  - 7. Final Note

1. Problem Framing & Scope

This exercise focuses on designing a key part of a record-to-record synchronization system between an internal system (System A) and an external CRM (System B).

The core challenge is to reliably synchronize individual record changes (CRUD) while handling schema differences and external API constraints such as rate limits.

For this submission, I intentionally implemented **only the core synchronization engine**, focusing on the outbound flow (Internal → External). The goal was to demonstrate clean design, extensibility, and clear responsibility boundaries rather than building a full production system.

2. High-Level Design

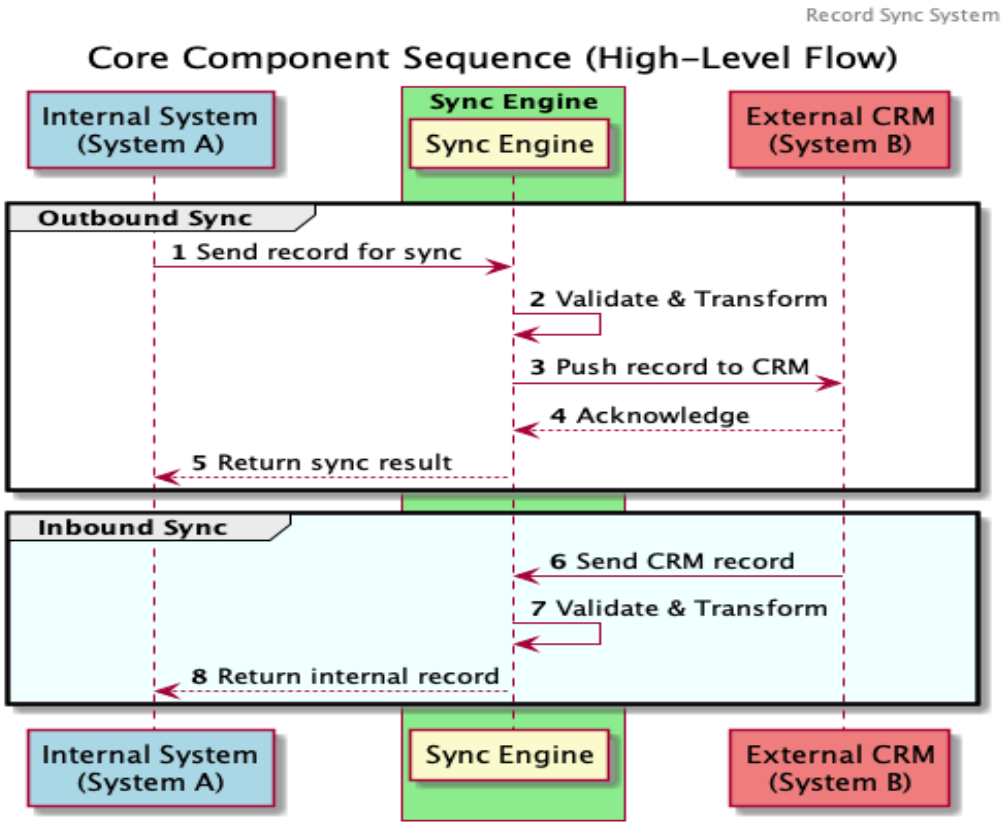
The solution is structured around a **Sync Engine** that sits between System A and the external CRM.

- **System A** produces record change events.
- **Sync Engine** orchestrates the synchronization logic.
- **External CRM** is accessed via a provider abstraction.

Inside the Sync Engine, responsibilities are deliberately separated:

- Validation ensures incoming records are structurally sound.
- Transformation converts internal records to the external schema.
- Rate limiting simulates external API constraints.
- CRM provider abstracts external API interactions.

This design keeps the engine focused on orchestration and avoids tight coupling with external systems.

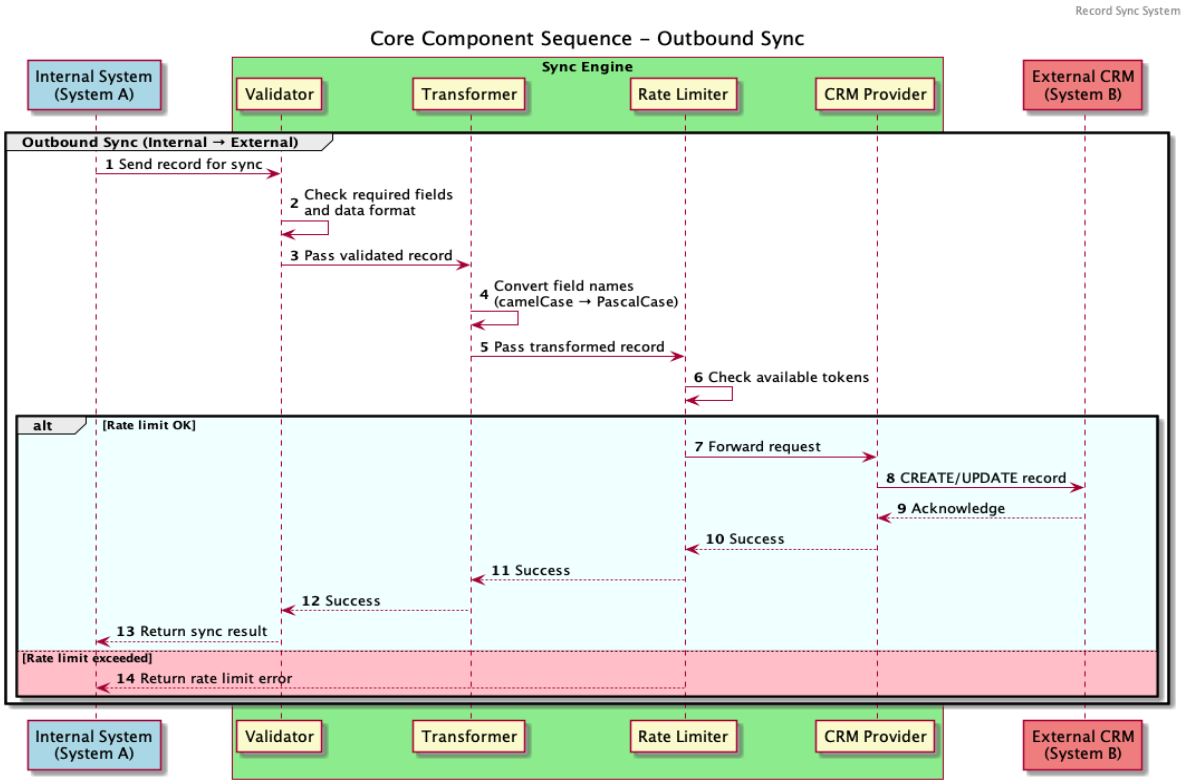


3. Core Synchronization Flow

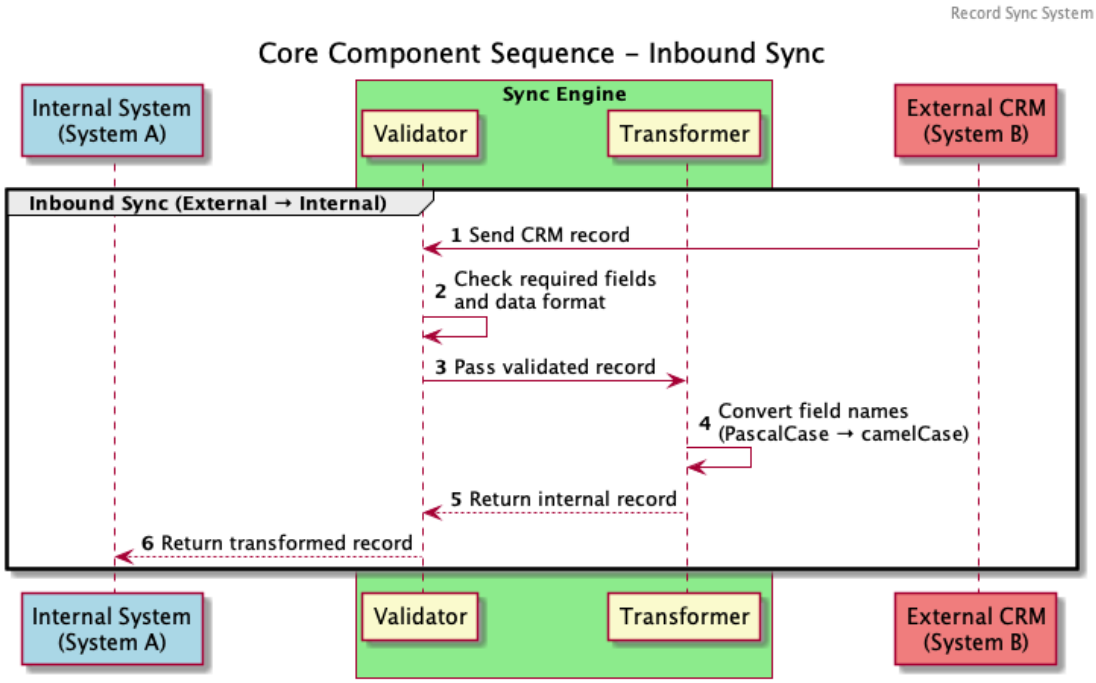
The outbound synchronization flow is intentionally linear and record-by-record:

1. System A submits a `SyncRequest`.
2. The Sync Engine validates the record.
3. The record is transformed into the external model.
4. A rate-limit check is performed.
5. The corresponding CRUD operation is executed on the CRM provider.
6. A `SyncResult` is returned to System A.

3.1 Outbound Sync



3.2 Inbound Sync



4. Key Design Decisions & Trade-offs

- **Synchronous processing** was chosen for simplicity and clarity. In a production environment, this would likely be asynchronous.
- **Token Bucket rate limiting** was used to approximate real SaaS API behavior, allowing short bursts while enforcing an average rate.
- **CRM abstraction via interface** allows additional providers to be added without modifying the Sync Engine.
- **In-memory components** (rate limiter, mocks) were used to keep the implementation lightweight and focused.

Trade-offs were made intentionally to avoid over-engineering while still demonstrating scalable design principles.

5. Assumptions & Future Enhancements

Assumptions

- Synchronization occurs record-by-record.
- Events are reliably delivered to the Sync Engine.
- Conflict resolution is handled outside this component.
- Rate limiting is per-instance and in-memory.
- Security, authentication, and persistence are out of scope.

Future Enhancements

- Distributed rate limiting (e.g., Redis-based).
- Async processing using message queues.
- Retry and backoff strategies.
- Observability (metrics and tracing).
- Support for additional CRM providers.

6. How to Run

1. **Main Class:** The project includes a simple `main` class (`SyncEngineDemo`) that demonstrates the synchronization flow.
2. **Unit tests:** are included for core components such as validation, transformation, rate limiting, and the sync engine.